

Instruction of Live Demonstration

Preparation

In this document, we are going to introduce how to perform the live demonstration for unsupervised learning task in INT104.

The following code block configures the Deepnote environment, which is not useful in the live demonstration as we are going to use PyCharm environment instead. The TAs will instruct you which specific Python environment to be used during the live demonstration. Another demonstration of using PyCharm will be provided in the tutorial session of week 10.

```
import subprocess
import sys
import csv
import numpy as np
import pandas as pd
from sklearn.metrics import pairwise_distances

def install(package):
    subprocess.check_call([sys.executable, "-m", "pip", "install", package])

install('openpyxl')
```

Requirement already satisfied: openpyxl in /root/venv/lib/python3.9/site-packages (3.1.5)
Requirement already satisfied: et-xmlfile in /root/venv/lib/python3.9/site-packages (from openpyxl) (2.0.0)

[notice] A new release of pip is available: 23.0.1 -> 25.0.1
[notice] To update, run: pip install --upgrade pip

On LMO, the following code will be provided as a py file. Over the demonstration session, you need to download the file into the computer and import the file into the PyCharm project.



```

def evaluate_clustering(X, labels, output_file='predicted_labels.csv'):
    """
    Evaluate the clustering result by calculating the ratio of intra-cluster distance
    to inter-cluster distance.

    Parameters:
    X : array-like, shape (n_samples, n_features)
        The input data.
    labels : array-like, shape (n_samples,)
        The cluster labels for each sample.

    Returns:
    float
        The ratio of intra-cluster distance to inter-cluster distance.
    """
    unique_labels = np.unique(labels)

    # Calculate intra-cluster distances
    intra_distances = []
    for label in unique_labels:
        cluster_points = X[labels == label]
        if len(cluster_points) > 1:
            intra_distance = np.mean(pairwise_distances(cluster_points))
            intra_distances.append(intra_distance)

    # Calculate inter-cluster distances
    inter_distances = []
    for i in range(len(unique_labels)):
        for j in range(i + 1, len(unique_labels)):
            cluster_i = X[labels == unique_labels[i]]
            cluster_j = X[labels == unique_labels[j]]
            inter_distance = np.mean(pairwise_distances(cluster_i, cluster_j))
            inter_distances.append(inter_distance)

    # Calculate the average intra-cluster and inter-cluster distances
    avg_intra_distance = np.mean(intra_distances) if intra_distances else 0
    avg_inter_distance = np.mean(inter_distances) if inter_distances else 1 # Avoid division by zero

    # Calculate the ratio
    ratio = avg_intra_distance / avg_inter_distance if avg_inter_distance != 0 else float('inf')

    # Save label information to a CSV file
    label_df = pd.DataFrame({'cluster_index': labels})
    label_df.to_csv(output_file, index=False)
    print("File saved successfully in", output_file)

    # Print key metric
    print(f"Intra-cluster to Inter-cluster distance ratio: {ratio:.4f}")

    return ratio

def evaluate_classification(predicted_labels, filename='predicted_labels.csv'):
    """
    Save predicted labels to a specified CSV file.

    Parameters:
    predicted_labels (list): A list of predicted labels to save.
    filename (str): The name of the CSV file where the labels will be saved.
    """
    try:
        with open(filename, mode='w', newline='') as file:
            writer = csv.writer(file)
            for label in predicted_labels:
                writer.writerow([label]) # Write each label in a new row
            print(f"Predicted labels successfully saved to {filename}.")
    except Exception as e:
        print(f"An error occurred while saving labels: {e}")

```

Live Demonstration - Unsupervised Learning

You then need to set up a new Python file or a set of new Python files to write your own Python script for performing unsupervised learning. At the end of Python script, you need to call the function "evaluate_clustering", which will print out the measure of ratio between intra-cluster to inter-cluster distance.

To make sure the file is properly run. Please make sure you have put the following files in the same folder (the working folder of your PyCharm):

- The testing dataset you have downloaded from LMO
- The Python script "evaluate.py" provided on LMO
- Your own Python script(s).

For the very first time run-through, please inform your TA to record your finished time. You then can keep on tuning your system. If you think you have tried your best, upload the obtained CSV file on LMO (please find the write portal for each algorithm).

```
# This is a file that demonstrates what you need to do over the live demonstration
# session for unsupervised learning. You need to:
# 1. Load the given data from the file.
# 2. Prepare the data for clustering.
# 3. Cluster the data.
# 4. Run the evaluation function.

import pandas as pd
from sklearn.cluster import KMeans

# You need the following command in your Python script:
# from evaluate import evaluate_clustering #

# Step 1: Read data from an Excel file
# Make sure to replace 'data.xlsx' with the path to your Excel file.
# Change the file name accordingly.
data = pd.read_excel('sample_data.xlsx')

# Step 2: Prepare the data for clustering
# Assuming the label information is in the first column and other columns
# contain feature information of samples.
# Adjust the column selection as necessary.
X = data.iloc[:, 1:].values # Use all columns except the last one for clustering

# Step 3: Perform K-means clustering
kmeans = KMeans(n_clusters=4, random_state=42)
labels = kmeans.fit_predict(X)

# Step 4: Evaluate the clustering results
# Remember to change the file name for different algorithms!
ratio = evaluate_clustering(X, labels, output_file='clustering_kmeans.csv')
```

File saved successfully in clustering_kmeans.csv
Intra-cluster to Inter-cluster distance ratio: 0.3204

Live Demonstration - Supervised Learning

You then need to set up a new Python file or a set of new Python files to write your own Python script for performing supervised learning. At the end of Python script, you need to call the function "evaluate_classification", which will save your file that could be uploaded via LMO.

To make sure the file is properly run. Please make sure you have put the following files in the same folder (the working folder of your PyCharm):

- The testing dataset you have downloaded from LMO
- The Python script "evaluate.py" provided on LMO
- Your own Python script(s).

For the very first time run-through, please inform your TA to record your finished time. You then can keep on tuning your system. If you think you have tried your best, upload the obtained CSV file on LMO (please find the write portal for each algorithm).

```
# For the live demonstration for the supervised learning, you need to:
# 1. Load the given data from the file.
# 2. Prepare the data.
# 3. Classify the samples
# 4. Run the evaluation function.

from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split

# You need the following command in your Python script:
# from evaluate import evaluate_classification

# Step 1 is the same with the previous sections hence will not
# be repeated.

# Step 2: Prepare the data for clustering
# Assuming the label information is in the first column and other columns
# contain feature information of samples.
# Adjust the column selection as necessary.
X2 = data.iloc[:, 1:].values # Use all columns except the last one for clustering
label_gt = data.iloc[:, 0].values

# Set up the training dataset and the testing dataset
X_train, X_test, y_train, y_test = train_test_split(X2, label_gt, test_size=0.2, random_state=42)

# Step 3: Train a classifier

knn = KNeighborsClassifier(n_neighbors=10)

knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

evaluate_classification(y_pred, "classification_knn.csv")
```

Predicted labels successfully saved to classification_knn.csv.